Customer Churn MLOps Pipeline: Complete Classroom Guide

Python, scikit-learn, MLflow, DVC, Git, GitHub Actions, pytest and FastAPI, built step by step in VS Code.

The original guide you pasted had broken code blocks (generate_data.py, validation.py, train.py, the tests and api.py were cut off). Every file below is complete and ready to type or paste. I also fixed two real problems: `data/customers.csv` must NOT be in `.gitignore` before `dvc add` (DVC refuses git-ignored paths), and a `pytest.ini` is added so plain `pytest` can import your modules.

Part A: Teaching plan

Session	Topic	Time	Students leave with
1	Setup + what is MLOps	60 min	Working VS Code, venv, folders
2	Data generation + validation	60 min	Clean dataset, validation script
3	Training + ROC-AUC + quality gate	75 min	Trained model, metrics.json
4	MLflow experiment tracking	45 min	Dashboard with 2+ compared runs
5	pytest + Git	60 min	5 passing tests, first commit
6	GitHub Actions CI	60 min	Green check on GitHub
7	DVC data versioning	60 min	Dataset tracked by DVC
8	FastAPI serving + wrap-up	60 min	Live /predict endpoint

Key message for day one: A model that scores well in a notebook is not production-ready. MLOps makes the work reproducible, testable, traceable and deployable.

The workflow students will build:

Customer data (CSV) → validation → training and evaluation → MLflow (experiments) → DVC (data versions) → pytest + GitHub Actions (automated checks and quality gate) → FastAPI (prediction endpoint)

Part B: Install the tools

1. **VS Code** from code.visualstudio.com, plus the Microsoft **Python** extension.
2. **Python 3.11** from python.org (tick "Add Python to PATH" on Windows).
3. **Git** from git-scm.com.
4. A free **GitHub** account.

Open VS Code, then Terminal → New Terminal, and check:

```
python --version
git --version
```

Python should say 3.11.x. On Windows, if `python` fails, try `py -3.11 --version`.

Part C: Create the project

```
mkdir Customer-Churn-Prediction-MLOps
cd Customer-Churn-Prediction-MLOps
python -m venv .venv
```

Activate the environment:

- **Windows PowerShell:** `.venv\Scripts\Activate.ps1`
- **Windows (if PowerShell blocks it):** `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` then run the line above again
- **Mac/Linux:** `source .venv/bin/activate`

You should see `(.venv)` at the start of the prompt. Then open the folder in VS Code (File → Open Folder) so the Explorer shows it.

Final folder structure

```
Customer-Churn-Prediction-MLOps/
├── .github/workflows/ml.yml
├── data/                  (customers.csv is generated)
├── artifacts/             (model + metrics are generated)
├── tests/test_pipeline.py
├── .gitignore
├── pytest.ini
├── requirements.txt
├── generate_data.py
├── validation.py
├── train.py
└── api.py
```

Create the empty folders `.github/workflows` and `tests` in the Explorer now. Files are added one at a time below.

requirements.txt

```
numpy==1.26.4
pandas==2.2.3
scikit-learn==1.5.2
joblib==1.4.2
mlflow==2.17.2
pytest==8.3.3
dvc==3.56.0
fastapi==0.115.4
uvicorn==0.32.0
```

```
python -m pip install --upgrade pip
pip install -r requirements.txt
python -c "import pandas, sklearn, mlflow, dvc, fastapi; print('All libraries
installed')"
```

Expected output: `All libraries installed`. Fix any install error before moving on.

Teaching tip: Pinned versions are themselves an MLOps lesson. Ask students: "Why not just write `pandas` without a version?" (Answer: a future release could break the pipeline or change results.)

Part D: Generate the dataset

We use synthetic data so no private customer information is needed.

Create **generate_data.py**:

```
from pathlib import Path

import numpy as np
import pandas as pd

SEED = 42
ROWS = 2000


def main():
    rng = np.random.default_rng(SEED)

    tenure = rng.integers(1, 73, size=ROWS)
    monthly_charges = np.round(
        rng.normal(70, 20, size=ROWS).clip(20, 130), 2
    )
    support_calls = rng.poisson(1.5, size=ROWS)
    contract = rng.choice(
        ["Monthly", "One Year", "Two Year"],
        size=ROWS,
        p=[0.55, 0.25, 0.20],
    )
    internet_service = rng.choice(
        ["DSL", "Fiber", "None"],
        size=ROWS,
        p=[0.35, 0.50, 0.15],
    )

    # A synthetic churn tendency, not a real-world rule.
    score = (
        -1.0
        - 0.035 * tenure
        + 0.018 * (monthly_charges - 70)
        + 0.38 * support_calls
        + 1.1 * (contract == "Monthly")
        - 0.7 * (contract == "Two Year")
        + 0.45 * (internet_service == "Fiber")
```

```
    )
    probability = 1 / (1 + np.exp(-score))
    churn = rng.binomial(1, probability)

    df = pd.DataFrame({
        "tenure": tenure,
        "monthly_charges": monthly_charges,
        "support_calls": support_calls,
        "contract": contract,
        "internet_service": internet_service,
        "churn": churn,
    })

    output = Path("data/customers.csv")
    output.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(output, index=False)

    print(f"Dataset created: {output}")
    print(f"Rows and columns: {df.shape}")
    print(f"Churn rate: {df['churn'].mean():.2%}")
    print(df.head())


if __name__ == "__main__":
    main()
```

```
python generate_data.py
```

Expected: a CSV with **2,000 rows and 6 columns**.

Column	Meaning
tenure	Months the customer has been with the company
monthly_charges	Monthly bill
support_calls	Number of support calls
contract	Monthly, One Year or Two Year
internet_service	DSL, Fiber or None
churn	1 = left, 0 = stayed (the **target**)

Explain to students: `churn` is the target variable; the other five columns are features. The data is synthetic and only for learning, never for real retention decisions. The fixed `SEED` makes the data identical on every machine (reproducibility).

Part E: Validate the data

Create **validation.py**:

```
from pathlib import Path

import pandas as pd

DATA_PATH = Path("data/customers.csv")

REQUIRED_COLUMNS = [
    "tenure",
    "monthly_charges",
    "support_calls",
    "contract",
    "internet_service",
    "churn",
]


def validate_data(file_path=DATA_PATH):
    file_path = Path(file_path)

    if not file_path.exists():
        raise FileNotFoundError(f"Dataset not found: {file_path}")

    df = pd.read_csv(file_path)

    if df.empty:
        raise ValueError("Dataset is empty")

    missing_columns = set(REQUIRED_COLUMNS) - set(df.columns)
    if missing_columns:
        raise ValueError(f"Missing columns: {sorted(missing_columns)}")

    if df[REQUIRED_COLUMNS].isnull().any().any():
        raise ValueError("Dataset contains missing values")
```

```
    if not df["churn"].isin([0, 1]).all():
        raise ValueError("Churn must contain only 0 and 1")

    if (df["tenure"] < 0).any():
        raise ValueError("Tenure cannot be negative")
    if (df["monthly_charges"] < 0).any():
        raise ValueError("Monthly charges cannot be negative")
    if (df["support_calls"] < 0).any():
        raise ValueError("Support calls cannot be negative")

    if not set(df["contract"].unique()).issubset(
        {"Monthly", "One Year", "Two Year"}
    ):
        raise ValueError("Unexpected contract type")

    if not set(df["internet_service"].unique()).issubset(
        {"DSL", "Fiber", "None"}
    ):
        raise ValueError("Unexpected internet service")

    if df["churn"].nunique() != 2:
        raise ValueError("Target must contain both churn classes")

    print(f"Data validation passed: {len(df)} rows")
    return df


if __name__ == "__main__":
    validate_data()
```

```
python validation.py
```

Expected: `Data validation passed: 2000 rows`.

Class activity: Open `data/customers.csv`, delete one value, save, and rerun validation. It should fail with a clear message. Restore the data with `python generate_data.py`.

MLOps concept: Data validation catches problems before they contaminate training.

Part F: Train the model

The script loads and validates data, splits it, preprocesses features, trains Logistic Regression, computes accuracy and ROC-AUC, logs everything to MLflow, saves the model and metrics, and fails if ROC-AUC is below **0.60** when `--check-quality-gate` is passed.

Create **train.py**:

```
import argparse
import json
from pathlib import Path

import joblib
import mlflow
import mlflow.sklearn
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from validation import validate_data

SEED = 42
MINIMUM_AUC = 0.60
TEST_SIZE = 0.20
MAX_ITER = 1000

NUMERIC_FEATURES = ["tenure", "monthly_charges", "support_calls"]
CATEGORICAL_FEATURES = ["contract", "internet_service"]
TARGET = "churn"


def build_model():
    preprocessor = ColumnTransformer([
        ("numeric", StandardScaler(), NUMERIC_FEATURES),
        ("categorical", OneHotEncoder(handle_unknown="ignore"),
         CATEGORICAL_FEATURES),
    ])
    return Pipeline([
        ("preprocess", preprocessor),
```

```
        ("classifier", LogisticRegression(
            max_iter=MAX_ITER, random_state=SEED
        )),
    ])


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--check-quality-gate", action="store_true")
    args = parser.parse_args()

    df = validate_data()

    X = df[NUMERIC_FEATURES + CATEGORICAL_FEATURES]
    y = df[TARGET]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=TEST_SIZE, random_state=SEED, stratify=y
    )

    Path("artifacts").mkdir(exist_ok=True)

    mlflow.set_experiment("Customer Churn Prediction")

    with mlflow.start_run() as run:
        model = build_model()
        model.fit(X_train, y_train)

        predictions = model.predict(X_test)
        probabilities = model.predict_proba(X_test)[:, 1]

        accuracy = accuracy_score(y_test, predictions)
        auc = roc_auc_score(y_test, probabilities)

        mlflow.log_params({
            "model_type": "LogisticRegression",
            "random_seed": SEED,
            "test_size": TEST_SIZE,
            "max_iter": MAX_ITER,
            "dataset_rows": len(df),
        })
        mlflow.log_metrics({
            "accuracy": float(accuracy),
```

```
            "auc": float(auc),
        })
        mlflow.sklearn.log_model(model, artifact_path="model")

        joblib.dump(model, "artifacts/model.joblib")

        metrics = {
            "accuracy": float(accuracy),
            "auc": float(auc),
            "minimum_auc": MINIMUM_AUC,
            "run_id": run.info.run_id,
            "rows": len(df),
        }
        with open("artifacts/metrics.json", "w") as file:
            json.dump(metrics, file, indent=4)

        print(f"Accuracy: {accuracy:.4f}")
        print(f"ROC-AUC: {auc:.4f}")
        print(f"MLflow run ID: {run.info.run_id}")

        if args.check_quality_gate and auc < MINIMUM_AUC:
            raise RuntimeError(
                f"Quality gate failed: ROC-AUC {auc:.4f} "
                f"is below {MINIMUM_AUC:.2f}"
            )

        print("Quality gate passed.")
        print("Model saved to artifacts/model.joblib")


if __name__ == "__main__":
    main()
```

```
python train.py
python train.py --check-quality-gate
```

Each command creates a new MLflow run. Your exact numbers may vary slightly; the script prints accuracy, ROC-AUC, the run ID and the gate status.

What ROC-AUC means (board explanation)

ROC-AUC measures how well the model **ranks** customers who churn above customers who do not.

- **0.50** is roughly random ranking.
- **1.00** is perfect ranking.
- **0.60** is our demonstration threshold.

Passing a threshold does not prove production readiness. Real systems need stronger validation, baselines and business-specific acceptance criteria.

Class activity (quality gate): Change `MINIMUM_AUC` to `0.99`, run `python train.py --check-quality-gate`, and watch it fail. Discuss why you should investigate a failure rather than lower the threshold to hide it. Then set it back to `0.60`.

Part G: Track experiments with MLflow

Start the dashboard in a **second terminal** (activate `.venv` there too):

```
mlflow ui --backend-store-uri ./mlruns --port 5000
```

Open http://127.0.0.1:5000 in the browser and click the **Customer Churn Prediction** experiment. Students should find parameters, accuracy, AUC and the logged model artifact.

Class activity (compare two runs):

1. In `train.py`, change `MAX_ITER` to `50`, or swap in `RandomForestClassifier` from `sklearn.ensemble` and change the `model_type` parameter text to match.
2. Run `python train.py`.
3. In MLflow, tick both runs and click **Compare**. Which has the better AUC?

Analogy: MLflow is the experiment record book. It answers: which model did we train, with which settings, and how well did it perform?

If the dashboard is empty, you started it from a different folder than the one you trained in.

Part H: Automated tests with pytest

Create **pytest.ini** in the project root:

```
[pytest]
pythonpath = .
```

Create **tests/test_pipeline.py**:

```
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import pytest

from validation import validate_data

DATA_PATH = Path("data/customers.csv")
MODEL_PATH = Path("artifacts/model.joblib")
METRICS_PATH = Path("artifacts/metrics.json")


def test_data_is_valid():
    df = validate_data(DATA_PATH)
    assert len(df) > 0
    assert "churn" in df.columns


def test_missing_values_are_rejected(tmp_path):
    df = pd.read_csv(DATA_PATH)
    df.loc[0, "tenure"] = np.nan
    bad_file = tmp_path / "bad_customers.csv"
    df.to_csv(bad_file, index=False)

    with pytest.raises(ValueError, match="missing values"):
        validate_data(bad_file)


def test_model_and_metrics_exist():
    assert MODEL_PATH.exists()
    assert METRICS_PATH.exists()


def test_model_prediction_shape():
    model = joblib.load(MODEL_PATH)
```

```
    sample = pd.DataFrame([{
        "tenure": 12,
        "monthly_charges": 80.0,
        "support_calls": 3,
        "contract": "Monthly",
        "internet_service": "Fiber",
    }])

    prediction = model.predict(sample)
    probability = model.predict_proba(sample)

    assert prediction.shape == (1,)
    assert prediction[0] in [0, 1]
    assert probability.shape == (1, 2)
    assert np.allclose(probability.sum(axis=1), 1.0)


def test_model_quality():
    with open(METRICS_PATH) as file:
        metrics = json.load(file)

    assert 0.0 <= metrics["auc"] <= 1.0
    assert metrics["auc"] >= metrics["minimum_auc"]
```

The tests assume you already ran `generate_data.py` and `train.py`.

```
pytest -v
```

Expected: **5 passed**. If one fails, read the error and fix the cause; do not delete the test.

The five tests cover: data validity, rejection of missing values, existence of model artifacts, prediction output shape, and the saved AUC threshold.

Part I: Git and .gitignore

Create **.gitignore**:

```
.venv/
__pycache__/
.pytest_cache/
*.pyc
.env

mlruns/
artifacts/
```

Important: We deliberately do not list `data/customers.csv` here. DVC (Part K) will create its own ignore rule for the CSV. Listing it now would make `dvc add` fail.

```
git init
git add .
git commit -m "Build initial customer churn MLOps pipeline"
```

If Git asks who you are:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Then repeat the commit. Before the next step, run `git rm --cached data/customers.csv` and commit again so the CSV is no longer tracked by Git (DVC will take over).

Part J: Automate with GitHub Actions

Create **.github/workflows/ml.yml**:

```
name: Customer Churn MLOps Pipeline

on:
  push:
  pull_request:

jobs:
  test-and-train:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
```

```
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip

      - name: Install dependencies
        run: python -m pip install -r requirements.txt

      - name: Generate sample data
        run: python generate_data.py

      - name: Validate data
        run: python validation.py

      - name: Train model and enforce quality gate
        run: python train.py --check-quality-gate

      - name: Run automated tests
        run: python -m pytest -v
```

Push to GitHub

1. On github.com create a new **empty** repository named `Customer-Churn-Prediction-MLOps` (no README).
2. In VS Code run (replace the URL with yours):

```
git add .
git commit -m "Add MLOps pipeline and automated tests"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/Customer-Churn-
Prediction-MLOps.git
git push -u origin main
```

If the remote already exists, use `git remote set-url origin YOUR_REPOSITORY_URL`.

Verify

Open the repository, click **Actions**. A green check means success; a red cross means a step failed, so open it and read the failing step.

Important distinction for students: this is continuous integration plus automated model validation. It does not deploy the model to a live service.

Class activity: Make the pipeline fail on purpose (set `MINIMUM_AUC` to `0.99`, push, watch the red cross), then fix it and push again.

Part K: Version the dataset with DVC

Git is great for code; DVC handles datasets. Git keeps a tiny pointer file, and DVC keeps the real data.

Step A: Initialize

```
dvc init
```

Commit any pending changes first if DVC complains.

Step B: Track the dataset

```
dvc add data/customers.csv
```

This creates `data/customers.csv.dvc` (the pointer) and `data/.gitignore` (so Git ignores the real CSV).

Step C: Local storage (classroom practice)

```
mkdir ..\dvc-storage
dvc remote add -d localremote ../dvc-storage
```

On Mac/Linux use `mkdir ../dvc-storage`. This folder is not cloud storage and GitHub Actions cannot see it. That is fine because CI regenerates the data.

Step D: Commit and push

```
git add .dvc/config data/customers.csv.dvc data/.gitignore .gitignore
git commit -m "Add DVC dataset versioning"
dvc push
git push
```

If a path is not found, check `git status` and `dvc status`.

Class activity (time travel): Change `ROWS = 3000` in `generate_data.py`, regenerate, run `dvc add data/customers.csv`, and commit the new pointer. Then run `git checkout HEAD~1 -- data/customers.csv.dvc` followed by `dvc checkout` to restore the old 2,000-row dataset. Students see that code, data version, dependencies and settings together reproduce an experiment.

Part L: Serve predictions with FastAPI

Create **api.py**:

```
from contextlib import asynccontextmanager
from pathlib import Path
from typing import Literal

import joblib
import pandas as pd
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

MODEL_PATH = Path("artifacts/model.joblib")
model = None


@asynccontextmanager
async def lifespan(app: FastAPI):
    global model
    if MODEL_PATH.exists():
        model = joblib.load(MODEL_PATH)
    yield


app = FastAPI(
    title="Customer Churn Prediction API",
    description="Predicts churn using the trained ML model.",
    version="1.0.0",
    lifespan=lifespan,
)


class Customer(BaseModel):
    tenure: int = Field(ge=0, le=120)
    monthly_charges: float = Field(ge=0)
```

```
    support_calls: int = Field(ge=0)
    contract: Literal["Monthly", "One Year", "Two Year"]
    internet_service: Literal["DSL", "Fiber", "None"]


@app.get("/")
def home():
    return {"message": "Customer Churn Prediction API is running"}


@app.get("/health")
def health():
    return {"status": "ok", "model_loaded": model is not None}


@app.post("/predict")
def predict(customer: Customer):
    if model is None:
        raise HTTPException(
            status_code=503,
            detail="Model not loaded. Run python train.py and restart the
API.",
        )

    data = pd.DataFrame([customer.model_dump()])
    prediction = int(model.predict(data)[0])
    probability = float(model.predict_proba(data)[0][1])

    return {
        "churn_prediction": prediction,
        "churn_probability": round(probability, 4),
        "label": "Likely to churn" if prediction == 1 else "Likely to stay",
    }
```

Run it:

```
uvicorn api:app --reload
```

Open http://127.0.0.1:8000/docs (the Swagger UI), choose **POST /predict**, click **Try it out**, and enter:

```
{
  "tenure": 12,
  "monthly_charges": 80.0,
  "support_calls": 3,
  "contract": "Monthly",
  "internet_service": "Fiber"
}
```

Click **Execute**. You get a prediction (0 or 1), a probability and a label. Try a long-tenure Two Year customer with few support calls and compare the probabilities.

Class activity: Send an invalid contract value such as `"Weekly"`. FastAPI rejects it with a 422 error. This is input validation at the serving layer, the same idea as Part E.

This is a local educational demo, not a secured production deployment.

Part M: Run everything from scratch (demo checklist)

Use this order whenever you demonstrate the full project:

1. Open the folder in VS Code and activate `.venv`
2. `pip install -r requirements.txt`
3. `python generate_data.py`
4. `python validation.py`
5. `python train.py`
6. `python train.py --check-quality-gate`
7. `mlflow ui --backend-store-uri ./mlruns --port 5000` and inspect the experiment
8. `pytest -v` and confirm 5 passed
9. `dvc init`, `dvc add data/customers.csv`, `dvc push`
10. `git push` and check the green tick under Actions
11. `uvicorn api:app --reload` and test `/predict` at `/docs`

Part N: Common errors and fixes

Error	What to do
`ModuleNotFoundError`	Activate `.venv`, then `pip install -r requirements.txt`
`Dataset not found`	Run `python generate_data.py` first
`artifacts/model.joblib` not found	Run `python train.py` before pytest or the API
`Quality gate failed`	Inspect AUC, data and features. Do not just lower the threshold
Tests cannot import `validation`	Make sure `pytest.ini` exists in the project root
MLflow dashboard empty	Start it from the same folder you trained in
`dvc add` says path is git-ignored	Remove `data/customers.csv` from `.gitignore`, run `git rm --cached data/customers.csv`, retry
Git push rejected	`git status`, inspect the remote, resolve remote changes first
DVC push fails	Check `dvc remote list` and that the storage folder exists
API returns 503	Train the model and restart the API
PowerShell blocks activation	`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

Part O: Concepts to emphasise in class

- **Reproducibility:** random seeds, virtual environments, pinned dependencies and Git recreate the same workflow.
- **Experiment tracking:** MLflow records parameters, metrics and model artifacts so runs can be compared.
- **Data versioning:** DVC ties a model to the exact dataset that trained it.
- **Automated quality control:** pytest checks the pipeline; the quality gate stops a weak model from passing CI.

- **CI/CD:** GitHub Actions runs the pipeline on every push. Automated deployment is the natural next stage.
- **Model serving:** FastAPI separates inference from training.

What real production MLOps adds

Model registry with approval and promotion, comparison against the current production model (a champion/challenger check, since our gate only uses a fixed AUC threshold), secure cloud storage, API authentication, drift and performance monitoring, and controlled deployment with rollback.

Part P: Assessment ideas

1. **Mini-quiz:** What does the quality gate protect against? Why is `MLruns/` not committed? What does a `.dvc` file contain?
2. **Lab task:** Replace Logistic Regression with Random Forest, log both runs in MLflow, and justify which to keep.
3. **Break-and-fix:** Introduce a data bug (negative tenure), show validation catching it, then show CI failing.
4. **Extension project:** Add a `/predict_batch` endpoint, or add a second quality gate on accuracy.

Recommended order for teaching: Python pipeline first, then MLflow, then tests and GitHub Actions, then DVC, and finally FastAPI. Students understand each tool better when they have already felt the problem it solves.

Part Q: What to push to GitHub (and what to keep out)

Push the Python source code, tests, configuration files and the GitHub Actions workflow. Keep your virtual environment, MLflow run files and generated model artifacts out of Git.

1. Files and folders to push

```
Customer-Churn-Prediction-MLOps/
│
├── .github/
│   └── workflows/
│       └── ml.yml                  PUSH
│
```

```
├── .dvc/
│   └── config                    PUSH (if using DVC)
│
├── data/
│   ├── customers.csv.dvc         PUSH (if using DVC)
│   └── .gitignore                PUSH (created by DVC)
│
├── tests/
│   └── test_pipeline.py          PUSH
│
├── .gitignore                    PUSH
├── pytest.ini                    PUSH
├── requirements.txt              PUSH
├── generate_data.py              PUSH
├── validation.py                 PUSH
├── train.py                      PUSH
├── api.py                        PUSH
└── README.md                     PUSH
```

`data/customers.csv.dvc`, `.dvc/config` and `data/.gitignore` are needed only if you have initialized DVC and configured its remote. Your DVC remote's storage folder (`../dvc-storage`) must stay **outside** the repository.

2. Files you should NOT push

Your `.gitignore` should contain:

```
.venv/
__pycache__/
.pytest_cache/
*.pyc
.env

mlruns/
artifacts/
```

File or folder	Why exclude it?
`.venv/`	Contains installed Python packages.
`mlruns/`	Contains local MLflow experiment data.

File or folder	Why exclude it?
`artifacts/`	Contains generated models and metrics; manage them separately if needed.
`data/customers.csv`	DVC manages the dataset version. DVC adds this to `data/.gitignore` for you when you run `dvc add`.
`.dvc/cache/`	Contains DVC's local cached data (DVC ignores it automatically).
`.env`	May contain secrets or credentials.

Note on `data/customers.csv`: Do not list it in the root `.gitignore` before running `dvc add`. DVC refuses to track a file that Git already ignores. After `dvc add`, DVC writes the ignore rule into `data/.gitignore` itself, which is why that small file must be pushed.

Important: Do not exclude `requirements.txt`, `pytest.ini`, `.github/workflows/ml.yml`, `tests/`, or your Python source files.

3. Push the files using VS Code

Open the terminal in your project folder and run:

```
git status
git add .
git status
git commit -m "Add complete MLOps pipeline"
git push
```

The second `git status` is a teaching moment: have students read the list of staged files and confirm that `.venv`, `mlruns` and `artifacts` do not appear.

If this is your first push and you have not connected a GitHub repository yet:

```
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/Customer-Churn-Prediction-MLOps.git
git push -u origin main
```

Replace `YOUR_USERNAME` with your actual GitHub username.

4. Verify the upload

After pushing, open your repository on GitHub and confirm that you can see:

- `generate_data.py`
- `validation.py`
- `train.py`
- `api.py`
- `requirements.txt`
- `pytest.ini`
- `tests/test_pipeline.py`
- `.github/workflows/ml.yml`

Then open the **Actions** tab and check whether the workflow passes.

5. One final detail about DVC and CI

The workflow in this guide regenerates the dataset with `generate_data.py`, so it does not need DVC. If you later change the workflow to use `dvc pull`, you must configure remote storage that GitHub's runners can reach (for example S3, Google Cloud Storage or Azure Blob) and add the required credentials as GitHub Secrets. The local DVC remote alone will not be available to GitHub's runners.